

Advanced Artificial Intelligence and Machine Learning

Sequential & Generative Models



Learning goals

- RNNs as the sequence-processing counterpart to CNNs' spatial processing: parameter sharing across time, and backpropagation through time
- LSTM and GRU cells as fixes for the vanishing-gradient problem in vanilla RNNs
- RNN application patterns: language modeling, word embeddings, encoder-decoder (seq2seq) architectures
- Autoencoders as unsupervised feature learners: the undercomplete bottleneck, the connection to PCA, and regularized variants (sparse, denoising, contractive)
- Variational Autoencoders (VAEs) as probabilistic generative models: the ELBO objective and the reparameterization trick, at the concept level

RNN: Motivation and Mechanics

MOTIVATION FOR RECURRENT NETWORKS

- The two architectures we've seen so far, fully-connected networks and CNNs, have input layers of fixed size and (typically) only handle fixed-length inputs.
- The primary reason: varying the size of the input layer would also vary the number of learnable weights.
- This is a problem for **sequence data** such as time series, audio, and text.
- **Recurrent Neural Networks (RNNs)** are a class of architectures that allow varying input lengths and properly account for the ordering in sequence data.

RNNS — INTRODUCTION

- Suppose we have text data and our task is to analyse its *sentiment*.
- Given an input sentence such as “This is good news.”, the network has to classify it as “positive” or “negative”.
- We would like to train a simple neural network (such as the one below) to perform the task.

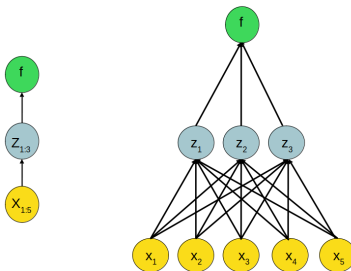


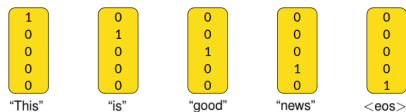
Figure: Two equivalent visualizations of a dense net with a single hidden layer, the left more abstract, showing the network from a layer point-of-view.

RNNS — INTRODUCTION

- Because sentences can be of varying lengths, we need to modify the dense net architecture to handle such a scenario.
- One approach draws inspiration from how a human reads a sentence: one word at a time.
- An important cognitive mechanism that makes this possible is “**short-term memory**”.
- As we read a sentence, we retain some information about the words already read and use this to understand the meaning of the entire sentence.
- Therefore, to feed the words in a sentence sequentially to a neural network, we need to give it the ability to retain some information about past inputs.

RNNS — INTRODUCTION

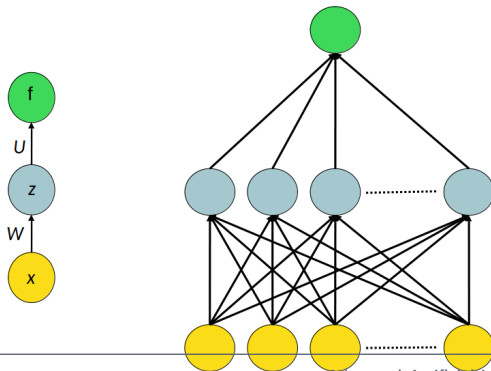
- When words in a sentence are fed to the network one at a time, the inputs are no longer independent: it is much more likely that “good” is followed by “morning” than by “plastic”. We need to model this **(long-term) dependency**.
- Each word must still be encoded as a fixed-length vector, because the input layer size stays fixed.
- Here, for visualization, each word is a one-hot vector of length 5 (<eos> = end of sequence).



While this is one option, the standard approach is word embeddings (more on this later).

RNNS — INTRODUCTION

- Our goal is to feed the words to the network sequentially, in discrete time-steps.
- A regular dense network with a single hidden layer has two sets of weights: input-to-hidden weights W and hidden-to-output weights U .



RNNS — INTRODUCTION

- To let the network retain information about past inputs, we introduce an **additional set of weights** V , from the hidden neurons at time-step t to the hidden neurons at time-step $t + 1$.
- This makes the hidden layer's activations depend on **both** the current input and the activations for the *previous* input.

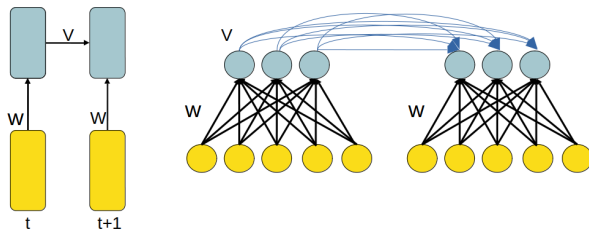


Figure: Input-to-hidden weights W and **hidden-to-hidden** weights V . Hidden-to-output weights U are omitted for readability.

RNNS — INTRODUCTION

- With this additional set of hidden-to-hidden weights V , the network is now a Recurrent Neural Network (RNN).
- In a regular feedforward network, the hidden layer's activations only use the input-hidden weights W (and bias b):

$$z = \sigma(W^T x + b)$$

- In an RNN, the activations at time-step t use *both* W and V :

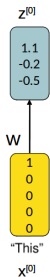
$$z^{[t]} = \sigma(V^T z^{[t-1]} + W^T x^{[t]} + b)$$

- $z^{[t]}$ represents the RNN's short-term memory: a function of the current input $x^{[t]}$ and the previous activations $z^{[t-1]}$.
- By recurrence, it contains a “summary” of *all* previous inputs.

RNN: Worked Example and Use Cases

APPLICATION EXAMPLE — SENTIMENT ANALYSIS

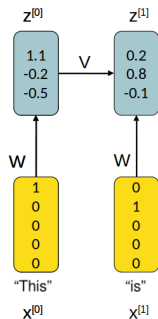
- At $t = 0$, we feed the word “This” to the network and obtain $z^{[0]}$.
- $z^{[0]} = \sigma(W^T x^{[0]} + b)$



Because this is the first input, there is no past state (equivalently, the state is initialized to 0).

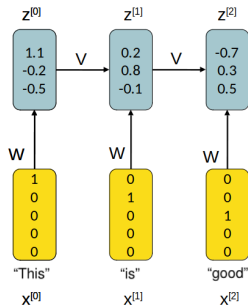
APPLICATION EXAMPLE — SENTIMENT ANALYSIS

- At $t = 1$, we feed the second word to obtain $z^{[1]}$.
- $z^{[1]} = \sigma(V^\top z^{[0]} + W^\top x^{[1]} + b)$



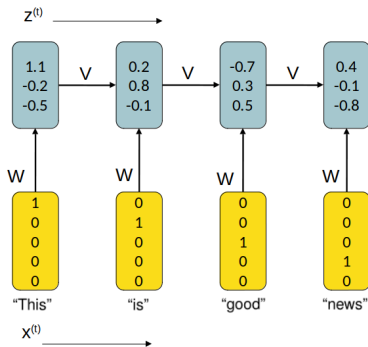
APPLICATION EXAMPLE — SENTIMENT ANALYSIS

- At $t = 2$, we feed the next word.
- $z^{[2]} = \sigma(V^\top z^{[1]} + W^\top x^{[2]} + b)$



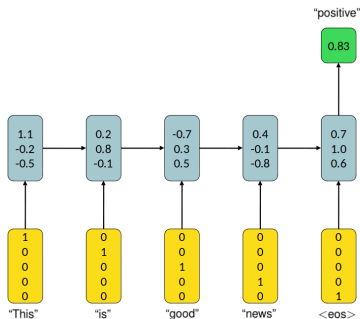
APPLICATION EXAMPLE — SENTIMENT ANALYSIS

- At $t = 3$, we feed the next word (“news”).
- $z^{[3]} = \sigma(V^\top z^{[2]} + W^\top x^{[3]} + b)$



APPLICATION EXAMPLE — SENTIMENT ANALYSIS

- Once the entire sequence has been processed, the prediction is generated by feeding the final time-step's activations to the output neuron(s).
- $f = \sigma(U^T z^{[4]} + c)$, where c is the output neuron's bias.



PARAMETER SHARING

- This way, the network processes the sentence one word at a time, and its length can vary based on the sequence length.
- No matter how long the input sequence is, the matrices W and V are the **same** at every time-step — another example of **parameter sharing** (recall this idea from CNNs, Session 5).
- Therefore, the number of weights in the network is independent of the input sequence length.

RNNS — USE-CASE-SPECIFIC ARCHITECTURES

RNNs are very versatile and apply to a wide range of tasks.

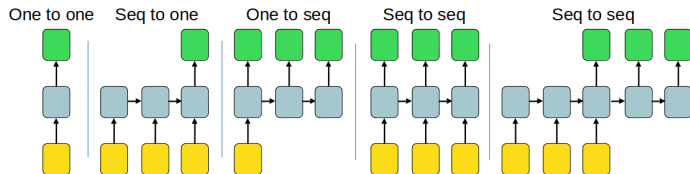


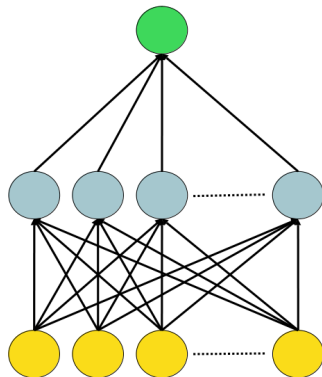
Figure: RNNs can be used in tasks with multiple inputs and/or multiple outputs.

Examples:

- Sequence-to-One: sentiment analysis, document classification.
- One-to-Sequence: image captioning.
- Sequence-to-Sequence: language modelling, machine translation, time-series prediction.

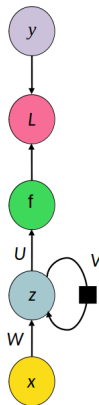
RNN: Computational Graph and Bidirectional RNNs

RNNS — COMPUTATIONAL GRAPH



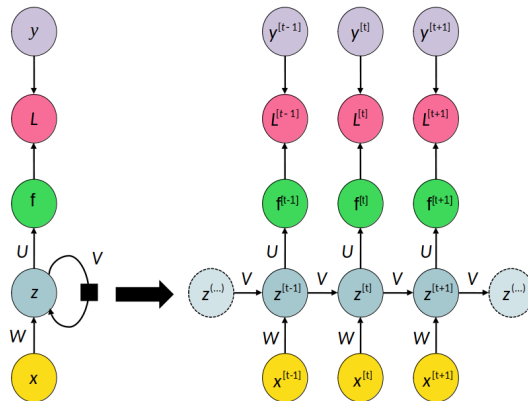
- On the left, an abstract representation of the computational graph for the network on the right. A loss L measures how far each output f is from the training target y .

RNNS — COMPUTATIONAL GRAPH



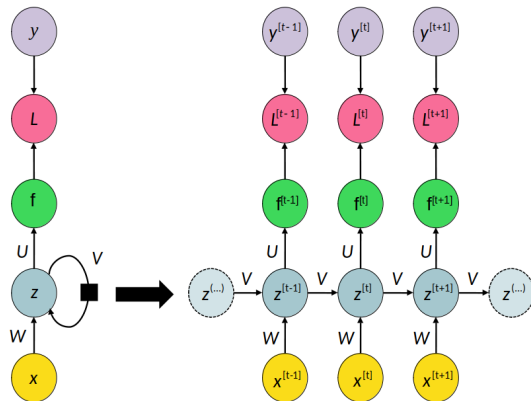
- A helpful way to think of an RNN: multiple copies of the same network, each passing a message to a successor.
- RNNs are networks with loops, allowing information to persist.

RNNS — COMPUTATIONAL GRAPH



- Things become clearer if we unfold the architecture.
- We call $z^{[t]}$ the *state* of the system at time t .
- The state contains information about the whole past sequence.

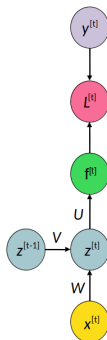
RNNS — COMPUTATIONAL GRAPH



- We went from

$$f = \tau(c + U^\top \sigma(b + W^\top x)) \text{ for the dense net, to}$$
$$f^{[t]} = \tau(c + U^\top \sigma(b + V^\top z^{[t-1]} + W^\top x^{[t]})) \text{ for the RNN.}$$

RNNS — COMPUTATIONAL GRAPH



- A potential computational graph for time-step t with

$$f^{[t]} = \tau(c + U^{\top} \sigma(b + V^{\top} z^{[t-1]} + W^{\top} x^{[t]}))$$

SEQ-TO-ONE MAPPINGS

RNNs do not need to produce an output at each time step — often only one output is produced after processing the whole sequence.

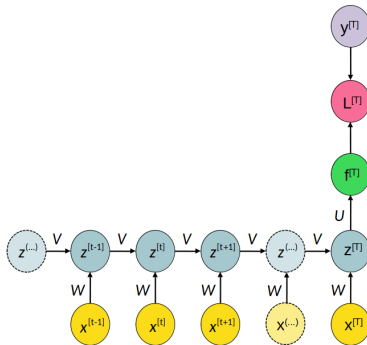
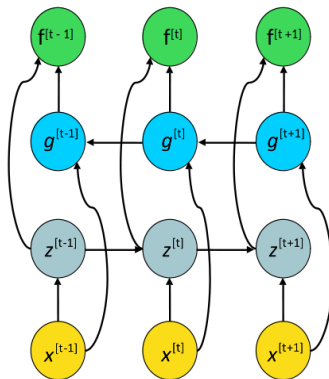


Figure: Time-unfolded RNN with a single output at the end of the sequence. Such a network can summarize a sequence into a fixed-size representation.

BIDIRECTIONAL RNNs

- A generalization of the simple RNN.
- Allows processing sequential data depending on both past and future inputs — e.g. predicting missing words, which likely depend on both preceding and following words.
- One RNN processes inputs forward, from $x^{[1]}$ to $x^{[T]}$, computing hidden states $(z^{[1]}, \dots, z^{[T]})$; another processes backward, from $x^{[T]}$ to $x^{[1]}$, computing $(g^{[T]}, \dots, g^{[1]})$.
- Predictions are based on both hidden states (e.g. by concatenation).
- With connections going back in time, the whole input sequence must be known in advance to train and infer from the model.
- Often used for the encoding of a sequence in machine translation. **Computational graph of a bidirectional RNN:**

BIDIRECTIONAL RNNs / 2



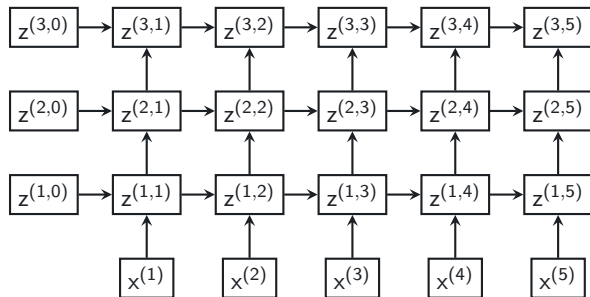
BIDIRECTIONAL RNNS / 3

Figure: A bidirectional RNN: a forward RNN processing inputs left to right, and a backward RNN processing inputs backwards in time.

MULTI-LAYER (STACKED) RNNs

- A stack of several RNNs (vanilla RNNs, LSTMs, GRUs, ...), each with its own parameters.
- The input vectors of the l -th RNN in the stack are the hidden states of the $(l - 1)$ -th RNN; the first RNN's inputs are the word embeddings, as usual.
- We can output the hidden states of the last RNN, or a combination (concatenation, average, ...) of the states of all RNNs.
- Stacking adds representational capacity, analogous to adding depth in a feedforward or convolutional net.

MULTI-LAYER (STACKED) RNNs / 2



RNN: Backpropagation Through Time

SIMPLE EXAMPLE: CHARACTER-LEVEL LANGUAGE MODEL

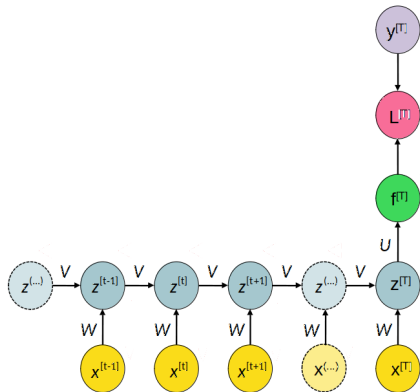
Task: learn a character probability distribution from input text.

- Suppose our vocabulary has four letters: “h”, “e”, “l”, “o”, and we train an RNN on the sequence “hello”.
- This training sequence is actually a source of 4 separate training examples:
 - “e” should be likely given the context of “h”
 - “l” should be likely given the context of “he”
 - “l” should also be likely given the context of “hel”
 - “o” should be likely given the context of “hell”

SIMPLE EXAMPLE: CHARACTER-LEVEL LANGUAGE MODEL / 2

The RNN has a 4-dimensional input and output. The exemplary hidden layer has 3 neurons. This diagram shows the forward-pass activations when the RNN is fed “hell”. The output holds the RNN’s confidences for the next character.

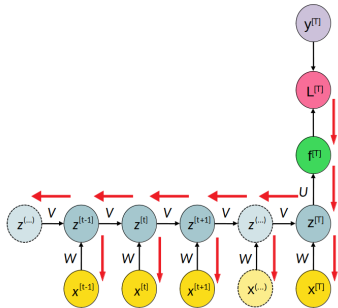
SIMPLE EXAMPLE: CHARACTER-LEVEL LANGUAGE MODEL / 3



SIMPLE EXAMPLE: CHARACTER-LEVEL LANGUAGE MODEL / 4

Our goal is to increase confidence for the correct letters and decrease it for all others (a softmax activation could squash these to probabilities). How do we train the network? **Backpropagation through time!**

BACKPROPAGATION THROUGH TIME



- To train the RNN, we need $\frac{dL}{du_{i,j}}$, $\frac{dL}{dv_{i,j}}$, and $\frac{dL}{dw_{i,j}}$.
- During backpropagation at time step t , we need to compute

$$\frac{dL}{dz^{[1]}} = \frac{dL}{dz^{[t]}} \frac{dz^{[t]}}{dz^{[t-1]}} \cdots \frac{dz^{[2]}}{dz^{[1]}}$$

LONG-TERM DEPENDENCIES

- Here, $z^{[t]} = \sigma(V^\top z^{[t-1]} + W^\top x^{[t]} + b)$, so:

$$\frac{dz^{[t]}}{dz^{[t-1]}} = \text{diag}(\sigma'(V^\top z^{[t-1]} + W^\top x^{[t]} + b))V^\top = D^{[t-1]}V^\top$$

$\vdots$

$$\frac{dL}{dz^{[1]}} = \frac{dL}{dz^{[t]}} D^{[t-1]} D^{[t-2]} \dots D^{[1]} (V^\top)^{t-1}$$

- In general, for an arbitrary past time-step $i < t$, $\frac{dz^{[t]}}{dz^{[i]}}$ contains the term $(V^\top)^{t-i}$ (by the chain rule).
- Based on the largest eigenvalue of $V^\top$, this term can cause vanishing or exploding gradients.

LONG-TERM DEPENDENCIES / 2

- This is more severe for RNNs than feedforward networks, because the **same** matrix V^T is multiplied repeatedly. As the gap between t and i grows, the instability worsens.
- It is thus challenging for RNNs to learn long-term dependencies: gradients either **vanish** (usually) or **explode** (rarely, but destructively) simply because errors propagate over very many stages.
- We can counteract exploding gradients with gradient clipping: clip the gradient norm at some threshold h :

$$\text{if } \|\nabla W\| > h : \nabla W \leftarrow \frac{h}{\|\nabla W\|} \nabla W$$

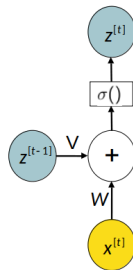
LONG-TERM DEPENDENCIES / 3

- Even for a stable RNN, long-term interactions get exponentially smaller weights than short-term ones — a more sophisticated fix is needed for vanishing gradients (next: LSTM, GRU).
- The vanishing gradient problem heavily depends on the activation function: sigmoid squashes into $[0, 1]$, so even large input changes yield small output changes, and gradients shrink further when layers stack. ReLU avoids this (gradients are 0 or 1, never saturating) at the cost of possible “dead” units.

LSTM and GRU

LONG SHORT-TERM MEMORY (LSTM)

The LSTM provides a way of dealing with vanishing gradients and modelling long-term dependencies.



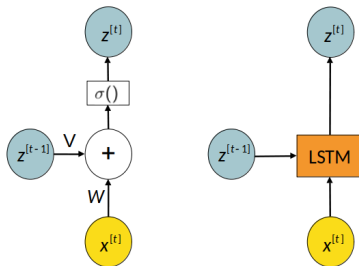
A simple RNN mechanism;

- Until now, we simply computed

$$z^{[t]} = \sigma(b + V^{\top} z^{[t-1]} + W^{\top} x^{[t]})$$

LONG SHORT-TERM MEMORY (LSTM)

The LSTM provides a way of dealing with vanishing gradients and modelling long-term dependencies.



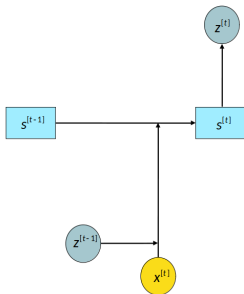
Left: A simple RNN mechanism; Right: An LSTM cell

- Until now, we simply computed

$$z^{[t]} = \sigma(b + V^{\top} z^{[t-1]} + W^{\top} x^{[t]})$$

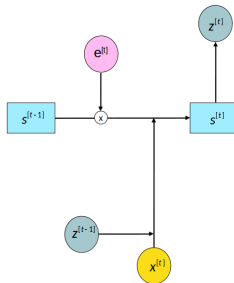
- Now we introduce the LSTM cell, a small network on its own.

LONG SHORT-TERM MEMORY (LSTM)



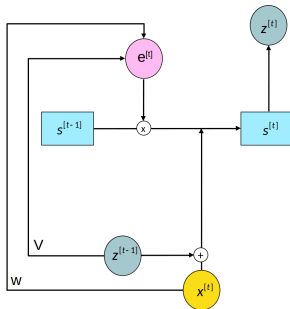
- The key to LSTMs is the **cell state** $s^{[t]}$.
- $s^{[t]}$ can be manipulated by different **gates** to forget old information, add new information, and read information out of it.
- Each gate is a vector the size of $s^{[t]}$, with elements between 0 (“let nothing pass”) and 1 (“let everything pass”).

LONG SHORT-TERM MEMORY (LSTM)



- **Forget gate $e^{[t]}$:** indicates which information of the old cell state we should forget.
- Intuition: a model predicting the next word might track the subject's gender for pronoun agreement. On seeing a new subject, we want to forget the old gender.

LONG SHORT-TERM MEMORY (LSTM)

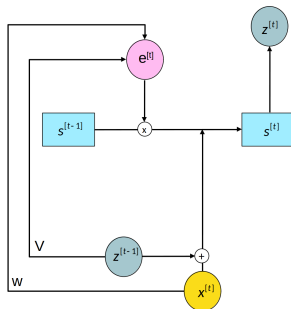


- We obtain the forget gate via

$$e^{[t]} = \sigma(b_e + V_e^\top z^{[t-1]} + W_e^\top x^{[t]})$$

- $\sigma()$ is a sigmoid, V_e, W_e are forget-gate-specific weights.

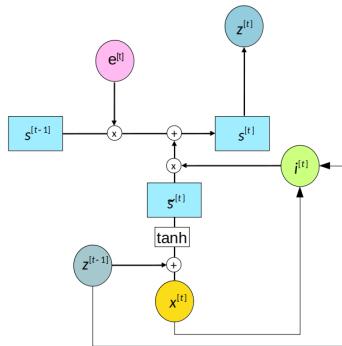
LONG SHORT-TERM MEMORY (LSTM)



- To compute $s^{[t]}$, first multiply (elementwise) the previous cell state $s^{[t-1]}$ by the forget gate:

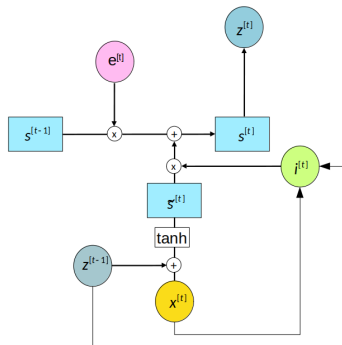
$$e^{[t]} \odot s^{[t-1]}, \text{ with } e^{[t]} \in [0, 1]$$

LONG SHORT-TERM MEMORY (LSTM)



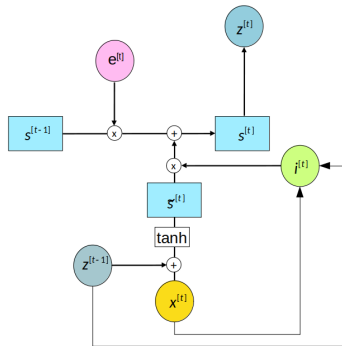
- **Input gate $i^{[t]}$:** indicates which new information should be added to $s^{[t]}$.
- Intuition: this is where we add the new subject's gender.

LONG SHORT-TERM MEMORY (LSTM)



- The new information is $\tilde{s}^{[t]} = \tanh(b + V^T z^{[t-1]} + W^T x^{[t]}) \in [-1, 1]$.
- The input gate is $i^{[t]} = \sigma(b_i + V_i^T z^{[t-1]} + W_i^T x^{[t]}) \in [0, 1]$.
- W, V are the new-information weights; W_i, V_i the input-gate weights.

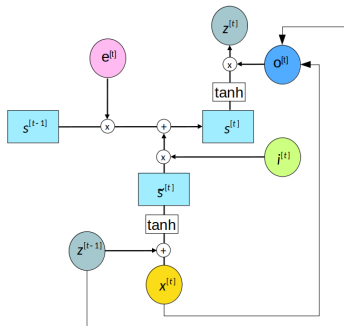
LONG SHORT-TERM MEMORY (LSTM)



- Now we can compute the cell state $s^{[t]}$:

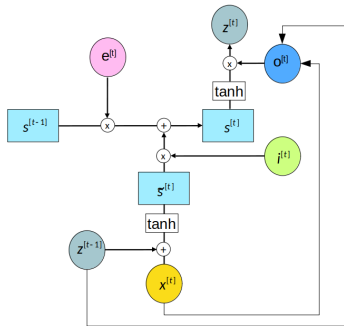
$$s^{[t]} = e^{[t]} \odot s^{[t-1]} + i^{[t]} \odot \tilde{s}^{[t]}$$

LONG SHORT-TERM MEMORY (LSTM)



- **Output gate $o^{[t]}$:** indicates which information from the cell state is filtered out.
- $o^{[t]} = \sigma(b_o + V_o^\top z^{[t-1]} + W_o^\top x^{[t]})$, with specific weights W_o, V_o .

LONG SHORT-TERM MEMORY (LSTM)



- Finally, the new state $z^{[t]}$ is a function of the cell state, multiplied by the output gate:

$$z^{[t]} = o^{[t]} \odot \tanh(s^{[t]})$$

GATED RECURRENT UNITS (GRU)

- The key distinction from regular RNNs: GRUs support gating of the hidden state.
- Dedicated mechanisms decide when a hidden state should be updated, and when it should be reset.
- These mechanisms are learned to:
 - avoid the vanishing/exploding gradient problem of a standard RNN,
 - using an update gate and a reset gate,
 - that control information flowing into (update gate) and out of (reset gate) memory.

GATED RECURRENT UNITS (GRU)

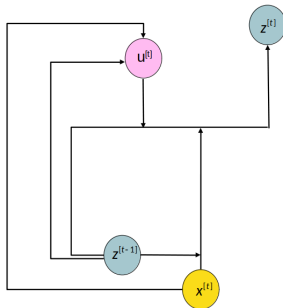


Figure: Update gate in a GRU.

- For time step t , the hidden state of the last time step is $z^{[t-1]}$. The update gate $u^{[t]}$ is:

GATED RECURRENT UNITS (GRU) / 2

- $u^{[t]} = \sigma(W_u^\top x^{[t]} + V_u^\top z^{[t-1]} + b_u)$
- We use a sigmoid to transform input values to $(0, 1)$.

GATED RECURRENT UNITS (GRU)

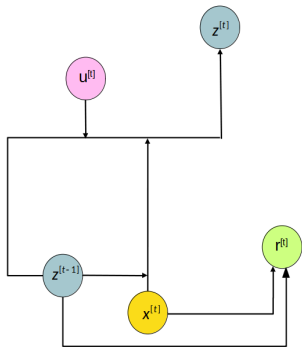


Figure: Reset gate in a GRU.

- Similarly, the reset gate $r^{[t]}$ is:

GATED RECURRENT UNITS (GRU) / 2

- $r^{[t]} = \sigma(W_r^\top x^{[t]} + V_r^\top z^{[t-1]} + b_r)$

GATED RECURRENT UNITS (GRU)

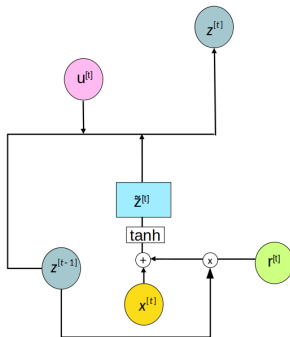


Figure: Hidden state computation in a GRU (elementwise multiplication).

- $\tilde{z}^{[t]} = \tanh(W_z^\top x^{[t]} + V_z^\top (r^{[t]} \odot z^{[t-1]}) + b_z).$

GATED RECURRENT UNITS (GRU) / 2

- In a conventional RNN, the hidden state update would instead be: $z^{[t]} = \tanh(W_z^\top x^{[t]} + V_z^\top z^{[t-1]} + b_z)$.

GATED RECURRENT UNITS (GRU)

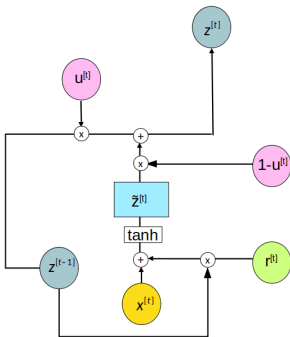


Figure: Update gate in a GRU (elementwise multiplication).

GATED RECURRENT UNITS (GRU) / 2

- The update gate $u^{[t]}$ determines how much of the old state $z^{[t-1]}$ and the new candidate state $\tilde{z}^{[t]}$ is used:
- $z^{[t]} = u^{[t]} \odot z^{[t-1]} + (1 - u^{[t]}) \odot \tilde{z}^{[t]}$.

These designs help eliminate the vanishing gradient problem in RNNs and capture dependencies over larger time-step distances. In summary, GRUs have two distinguishing features:

- Reset gates help capture short-term dependencies.
- Update gates help capture long-term dependencies.

GRU VS. LSTM

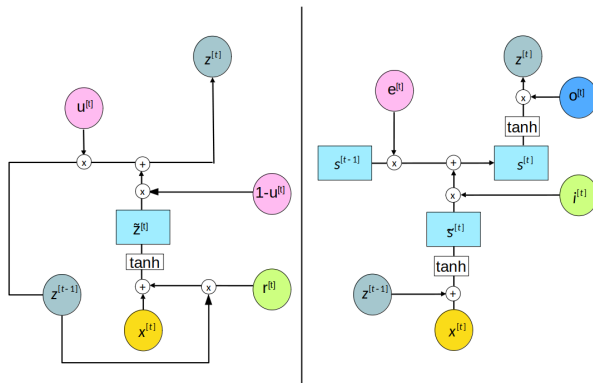
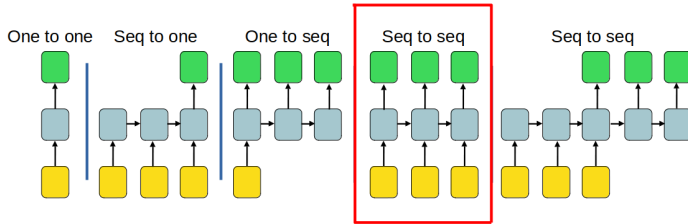


Figure: GRU vs. LSTM

RNN Applications

Seq-to-Seq (Type I)



RNNS — LANGUAGE MODELLING

- We already built a “sequence-to-one” RNN model for sentiment analysis.
- Another common NLP task is **language modelling**.
- Input: word/character, one-hot encoded.
- Output: probability distribution over words/characters given previous words:

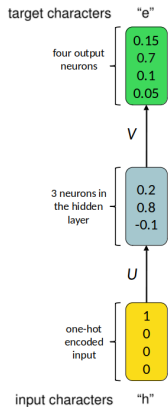
$$\mathbb{P}(y^{[1]}, \dots, y^{[T]}) = \prod_{i=1}^T \mathbb{P}(y^{[i]} | y^{[1]}, \dots, y^{[i-1]})$$

→ given previous characters, the RNN models the probability distribution of the next character.

RNNS — LANGUAGE MODELLING

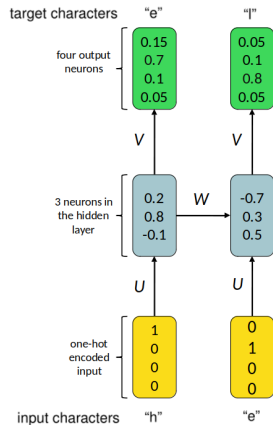
- We feed the characters in “hello” one at a time to a “seq-to-seq” RNN.
- “h”, “e”, “l”, “o” are one-hot coded as length-4 vectors (ignoring <eos>); the output layer has 4 neurons, one per character.
- At each time step, the RNN outputs a softmax probability distribution over the 4 possible next characters.
- If trained on English text:
 - “e” should be likely given “h”.
 - “l” should be likely given “he”.
 - “l” should **also** be likely given “hel”.
 - and “o” should be likely given “hell”.

RNNS — LANGUAGE MODELLING



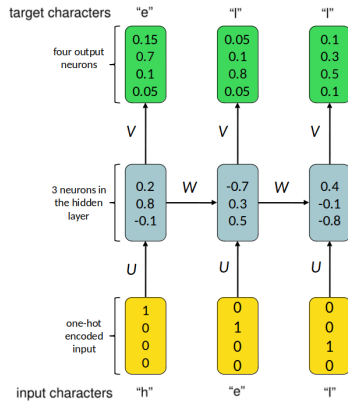
The probability of “e” should be high, given the context of “h”.

RNNS — LANGUAGE MODELLING



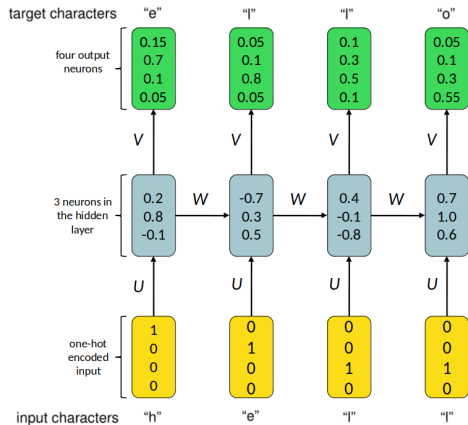
The probability of "l" should be high, given the context of "he".

RNNS — LANGUAGE MODELLING



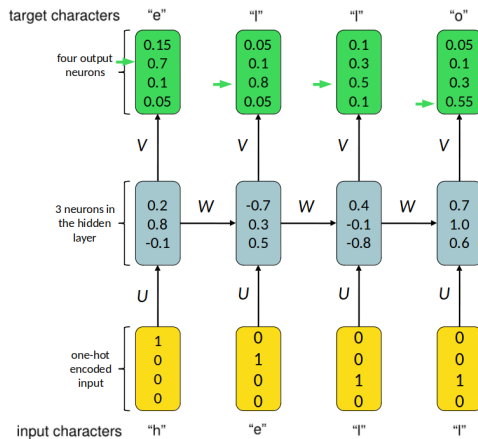
The probability of "l" should **also** be high, given the context of "hel".

RNNS — LANGUAGE MODELLING



The probability of "o" should be high, given the context of "hell".

RNNS — LANGUAGE MODELLING



During training, our goal is to increase confidence for the correct letters (green arrows) and decrease it for all others.

WORD EMBEDDINGS

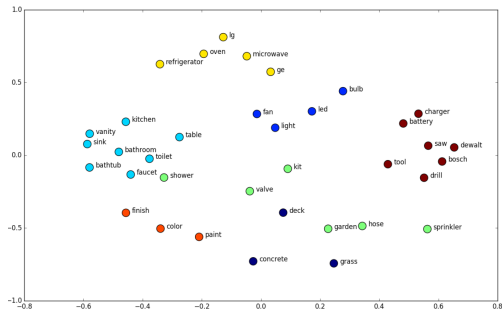


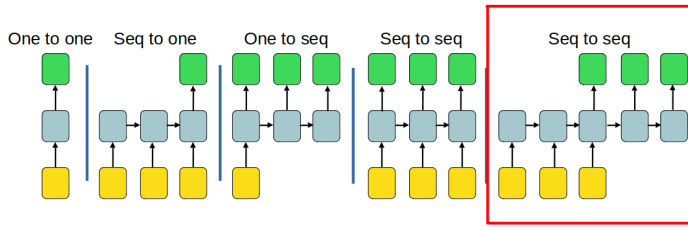
Figure: Two-dimensional embedding space. The embedding space is typically much higher-dimensional (Ruizendaal, 2018).

- Instead of one-hot representations, it is standard practice to encode each word as a dense (not sparse) vector of fixed size, capturing its underlying semantic content.
- Similar words are embedded close to each other in a lower-dimensional embedding space.

WORD EMBEDDINGS / 2

- The embedding dimensionality is typically much smaller than the dictionary size.
- Using them gives a “warm start” for any NLP task — an easy way to incorporate prior knowledge, and a rudimentary form of **transfer learning**.
- Two popular approaches to learn word embeddings: **word2vec** (Google) and **GloVe** (Stanford). Typically 100 to 1000 dimensional.
- These embeddings capture meaning to an extent, but not the *context*-dependent semantics of a word, since each word has one static precomputed representation — e.g. “bank” could mean a financial institution or a river bank.

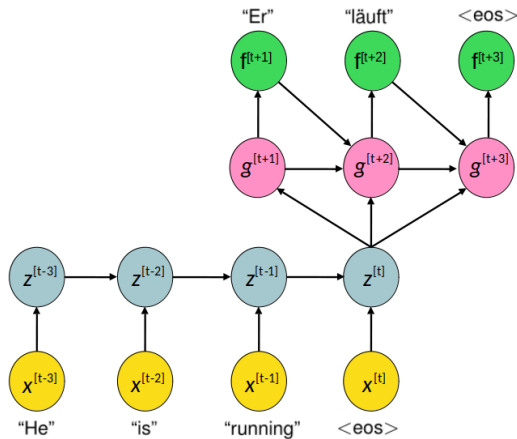
Seq-to-Seq (Type II)



ENCODER-DECODER NETWORK

- For applications such as question answering, dialogue systems, or machine translation, the network needs to map an input sequence to an output sequence of **different length**.
- This is what an encoder-decoder (a.k.a. sequence-to-sequence) architecture enables.

ENCODER-DECODER NETWORK / 2



ENCODER-DECODER NETWORK / 3

Figure: Encoder-decoder allows an RNN to process different-length input and output sequences. Information from the input is encoded in the context vector (here, the final hidden state), passed to every decoder hidden state, which produces the target sequence.

- An encoder RNN processes the input sequence (length n_x) and computes a fixed-length context vector C , usually the final hidden state or a simple function of the hidden states.
- One time step at a time, information from the input is added to the hidden state and passed forward through the encoder's recurrent connections.
- C summarizes important information from the input — e.g. the intent of a question, or the meaning of a text to translate.

ENCODER-DECODER NETWORK / 4

- The decoder RNN uses this to predict the output sequence (length n_y , which may differ from n_x).
- In machine translation, the decoder is a language model with recurrent connections between the output at one time step and the hidden state at the next, as well as between hidden states:

$$\mathbb{P}(y^{[1]}, \dots, y^{[n_y]} | \mathbf{x}^{[1]}, \dots, \mathbf{x}^{[n_x]}) = \prod_{t=1}^{n_y} p(y^{[t]} | C; y^{[1]}, \dots, y^{[t-1]})$$

- Trained jointly to minimize the translation error given a source sentence.

ENCODER-DECODER NETWORK / 5

- Each conditional probability is

$$p(y^{[t]}|y^{[1]}, \dots, y^{[t-1]}; C) = f(y^{[t-1]}, g^{[t]}, C)$$

where f is a non-linear function (e.g. tanh) and $g^{[t]}$ is the decoder's hidden state.

REFERENCES



Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2014). *Show and Tell: A Neural Image Caption Generator*.



Ruizendaal, R. (2018, October 21). *Using deep learning for structured data with entity embeddings*. Medium. <https://towardsdatascience.com/deep-learning-structured-data-8d6a278f3088>



Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*.

Unsupervised Learning

UNSUPERVISED LEARNING

- So far, we have applied neural networks to **supervised learning**: labeled data $(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), \dots, (\mathbf{x}^{(n)}, \mathbf{y}^{(n)})$, exploited to learn a function mapping $\mathbf{x}$ to $\mathbf{y}$ (classification, regression, ...).
- In **unsupervised learning**, training data consists of unlabeled input points $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$, and the goal is to learn some underlying hidden structure of the data.

UNSUPERVISED LEARNING / 2

Two motivating examples, both central to deep learning:

Feature extraction / representation learning: features learned from unlabeled data improve performance in a supervised (or semi-supervised) setting. This is exactly what an **autoencoder** does — covered next.

Density fitting / generative modeling: learn p_{data} itself, to generate new data, detect outliers, or reconstruct missing/corrupted inputs. This is what a **Variational Autoencoder** does — covered later in this session.

Autoencoders: Basic Principle

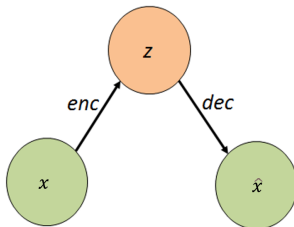
AUTOENCODER — TASK AND STRUCTURE

- Autoencoders (AEs) are NNs for unsupervised learning of a good, lower-dimensional **representation** z (the **code**) from unlabeled training data.
- An AE consists of two parts:
 - **encoder**: maps x to the low-dimensional latent variable $z = enc(x)$.
 - **decoder**: maps z back to a reconstruction $\hat{x} = dec(z)$ of x .
- The loss uses no labels, measuring reconstruction quality against the input:

$$L(x, dec(enc(x)))$$

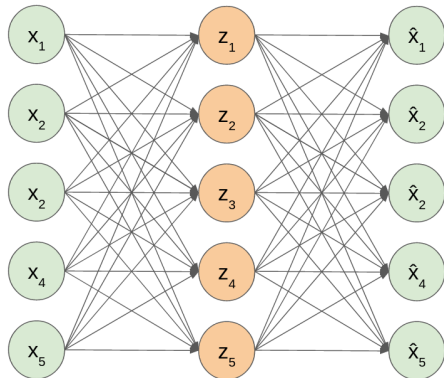
AUTOENCODER — TASK AND STRUCTURE / 2

The general structure of an AE as a computational graph:



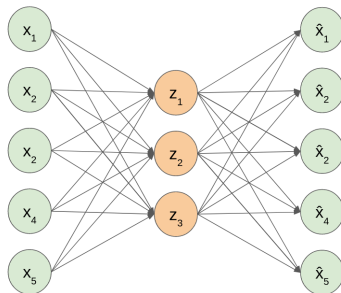
UNDERCOMPLETE AUTOENCODERS

- A naive AE would simply learn the identity $dec(enc(x)) = \hat{x}$ — not useful.



UNDERCOMPLETE AUTOENCODERS

- Therefore we introduce a “bottleneck” layer: restrict the architecture such that $\dim(z) < \dim(x)$ — such an AE is called **undercomplete**.
- With fewer hidden neurons than input neurons, it is forced to capture only the most salient features, learning a “compressed” representation of the input.



EXPERIMENT: LEARN TO ENCODE

MNIST

- Compress MNIST: train undercomplete AEs with different dimensions of the internal representation z (different “bottleneck” sizes).



Figure: Top row: original digits; bottom row: reconstructions.

- $\dim(z) = 784 = \dim(x)$.

EXPERIMENT: LEARN TO ENCODE

MNIST

- Compress MNIST: train undercomplete AEs with different dimensions of the internal representation z (different “bottleneck” sizes).



Figure: Top row: original digits; bottom row: reconstructions.

- $\dim(z) = 64$.

EXPERIMENT: LEARN TO ENCODE

MNIST

- Compress MNIST: train undercomplete AEs with different dimensions of the internal representation z (different “bottleneck” sizes).

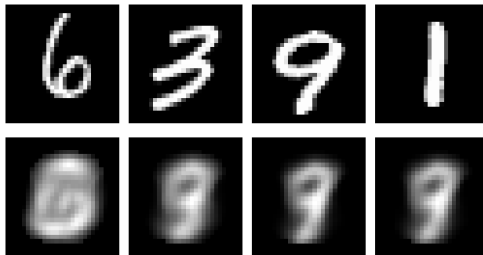
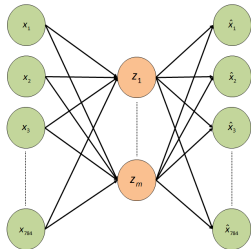


Figure: Top row: original digits; bottom row: reconstructions.

- $\dim(z) = 1$.

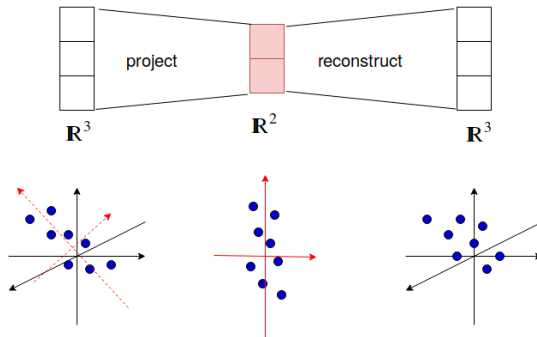
AES AS PRINCIPAL COMPONENT ANALYSIS

- Consider an undercomplete AE with a **linear** encoder and a **linear** decoder.
- Using the L2-loss $\|x - dec(enc(x))\|_2^2$ with inputs normalized to zero mean, we want the **linear projection** of the data with minimal L2-reconstruction error.



AES AS PRINCIPAL COMPONENT ANALYSIS / 2

- It can be shown that the optimal solution is an **orthogonal** linear transformation (a rotation of the coordinate system), given by the $\dim(z) = k$ singular vectors with the largest singular values.



AES AS PRINCIPAL COMPONENT ANALYSIS / 3

- This is equivalent to **Principal Component Analysis (PCA)**: an orthogonal transformation converting correlated variables into linearly uncorrelated **principal components**, with the first component having the largest possible variance.
- The two formulations are equivalent: “Find a linear projection into a k -dimensional space that ...”
 - “... minimizes the L2-reconstruction error” (AE-based).
 - “... maximizes the variance of the projected datapoints” (statistical).
- An AE with a non-linear decoder/encoder is a non-linear generalization of PCA.

AES AS PRINCIPAL COMPONENT ANALYSIS / 4

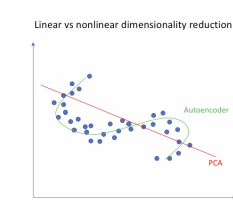


Figure: AEs can learn nonlinear manifolds (Jordan, 2018).

Regularized Autoencoders

OVERCOMPLETE AE — PROBLEM

Overcomplete AE (code dimension $\geq$ input dimension): even a linear AE can copy the input to the output without learning anything useful. How can an overcomplete AE be useful?

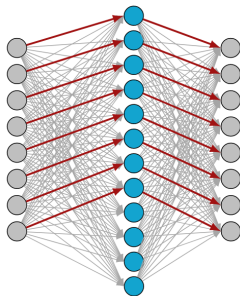


Figure: Overcomplete AE that learned to copy its input to the hidden layer, then to the output (Ponti et al., 2016).

OVERCOMPLETE AE — PROBLEM / 2

- Goal: choose code dimension and encoder/decoder capacity based on the problem.
- **Regularized AEs** modify the loss function to prevent trivially copying the input, and to encourage additional properties.
- Examples:
 - **Sparse AE**: sparsity of the representation.
 - **Denoising AE**: robustness to noise.

⇒ A regularized AE can be overcomplete and nonlinear, but still learn something useful about the data distribution.

SPARSE AUTOENCODER

- **Idea:** regularize with a sparsity constraint:

$$L(x, dec(enc(x))) + \lambda \|z\|_1$$

- Keeps the number of active neurons per training input low, forcing the model to respond to unique statistical features of the input.
- **Application:** beyond classic feature learning for downstream tasks, sparse AEs are now used to decompose the internal activations of trained neural networks (e.g. LLMs) into human-interpretable, non-overlapping features — a key tool in mechanistic interpretability research (Anthropic, 2024).

SPARSE AUTOENCODER / 2

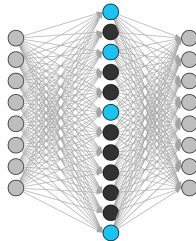


Figure: Sparse Autoencoder (Ponti et al., 2016).

DENOISING AUTOENCODERS (DAE)

The denoising autoencoder (DAE) receives a corrupted data point as input and is trained to predict the original, uncorrupted data point.

- Idea: the representation should be robust to noise.
- Produce a corrupted version $\tilde{x}$ of x , e.g. by
 - random assignment of a subset of inputs to 0,
 - adding Gaussian noise.
- Modified reconstruction loss: $L(x, \text{dec}(\text{enc}(\tilde{x}))) \rightarrow$ denoising AEs must learn to undo this corruption.

DENOISING AUTOENCODERS (DAE) / 2

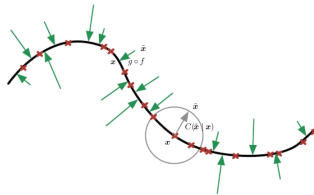


Figure: Denoising autoencoders — “manifold perspective” (Goodfellow et al., 2016).

- A DAE is trained to map a corrupted point $\tilde{x}$ back to the original x : the vector $dec(enc(\tilde{x})) - \tilde{x}$ points approximately towards the nearest point on the data manifold.
- Thus, the DAE learns a vector field pointing back onto the manifold, indicated by the green arrows.

EXPERIMENT: ENCODE MNIST WITH A DAE

- We corrupt the MNIST data with Gaussian noise and then try to denoise it as well as possible.
- To corrupt the input, we randomly add or subtract uniformly-distributed values to each image entry.

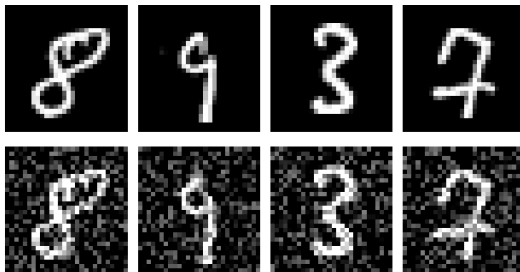


Figure: Top row: original data; bottom row: corrupted MNIST data.

EXPERIMENT: ENCODE MNIST WITH A DAE

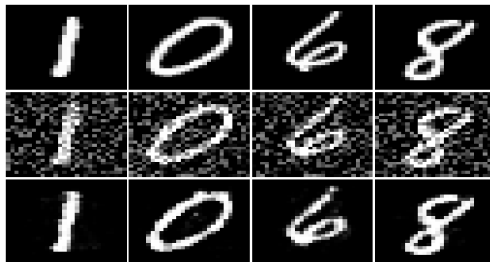


Figure: Top row: original digits; middle: corrupted; bottom: denoised/reconstructed (prediction).

- $\dim(\mathbf{z}) = 1568$ (overcomplete).

EXPERIMENT: ENCODE MNIST WITH A DAE

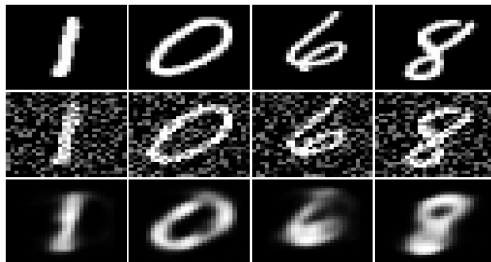


Figure: Top row: original digits; middle: corrupted; bottom: denoised/reconstructed (prediction).

- $\dim(\mathbf{z}) = 8$.

REFERENCES

-  Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
-  Jordan, J. (2018, March 19). *Introduction to autoencoders*.
<https://www.jeremyjordan.me/autoencoders/>
-  Ponti, M. A., Ribeiro, L. S. F., Nazare, T. S., Bui, T., & Collomosse, J. (2017). Everything You Wanted to Know about Deep Learning for Computer Vision but Were Afraid to Ask. *2017 30th SIBGRAPI Conference on Graphics, Patterns and Images Tutorials*, 17–41.
-  Anthropic. (2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. Transformer Circuits Thread.
<https://transformer-circuits.pub/2024/scaling-monosemanticity/>

Variational Autoencoders (VAE)

VARIATIONAL AUTOENCODER (VAE): INTUITION

Independently proposed by:

- Kingma and Welling, *Auto-Encoding Variational Bayes*, ICLR 2014
- Rezende, Mohamed and Wierstra, *Stochastic back-propagation and variational inference in deep latent Gaussian models*. ICML 2014

Conventional AEs compute a deterministic feature vector describing the input's attributes in latent space:

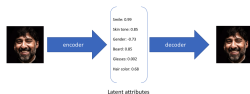


Figure: Source: Jordan (2018)

- The key difference in VAEs: they use a **variational** approach to learn the latent representation, allowing observations in latent space to be described **probabilistically**.

VARIATIONAL AUTOENCODER (VAE): INTUITION

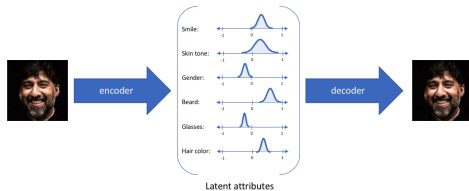


Figure: Source: Jordan (2018)

- Describe each latent attribute as a probability distribution.
- This allows modelling uncertainty in the input data.

VAE: STATISTICAL MOTIVATION

- Suppose a hidden variable z generates an observation x .
- By training the VAE, we want to determine the distribution of z and compute $p(z|x)$:

$$p(z|x) = \frac{p(x|z) p(z)}{p(x)}$$

- Computing $p(x)$ is difficult, since it is an intractable integral:

$$p(x) = \int p(x|z) p(z) dz$$

- Instead, we apply **variational inference**: approximate $p(z|x)$ by a tractable distribution $q(z|x)$ that is very similar to it.

VAE: STATISTICAL MOTIVATION

- KL divergence measures the difference between two distributions. To make $q(z|x)$ similar to $p(z|x)$, we could minimize

$$\min KL(q(z|x) || p(z|x))$$

- This is equivalent to maximizing:

$$\max E_{q(z|x)} \log p(x|z) - KL(q(z|x) || p(z))$$

- The first term is the reconstruction likelihood; the second ensures the learned q stays close to the true prior $p(z)$.

VAE: STATISTICAL MOTIVATION

- $p(z)$ is often assumed Gaussian $\rightarrow$ determining $q(z|x)$ reduces to estimating μ and σ .
- We use a neural network to estimate $q(z|x)$ (the encoder) and the decoder network to model $p(x|z)$.

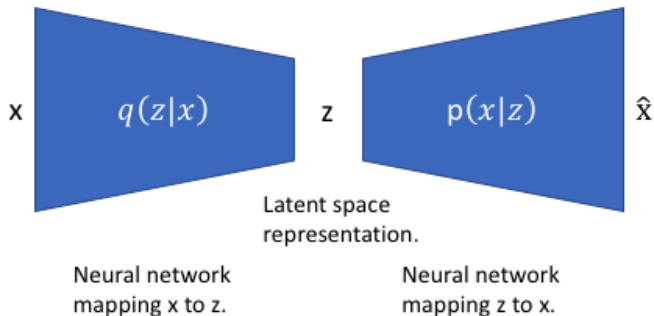


Figure: Source: Jordan (2018)

VAE TRAINING

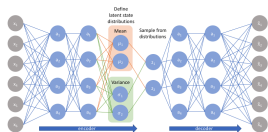


Figure: Source: Jordan (2018)

- Loss function (the **Evidence Lower BOund**, or **ELBO**):

$$L(\theta, \phi; x, z) = \underbrace{E_{q_{\phi}(z|x)} \log p_{\theta}(x|z)}_{\text{reconstruction term}} - \underbrace{KL(q_{\phi}(z|x) || p_{\theta}(z))}_{\text{stay-close-to-prior term}}$$

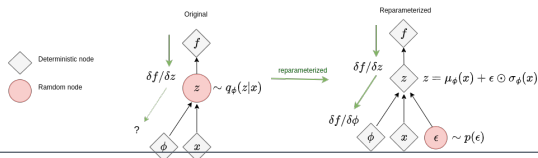
- Maximizing the ELBO jointly maximizes reconstruction quality and keeps the encoder's distribution close to the prior.
- Problem: the network contains a **sampling** step → we cannot backpropagate through it!

REPARAMETERIZATION TRICK

- We cannot backpropagate through random sampling — what now?
- Idea: “push” the random sampling out of the backpropagation path by **reparameterizing** it.
- Instead of sampling z directly from $\mathcal{N}(\mu_z(x), \sigma_z^2(x))$, we sample a fixed noise source $\epsilon \sim \mathcal{N}(0, 1)$ and compute

$$z = \mu_z(x) + \sigma_z(x) \odot \epsilon$$

- Now the only randomness is in ϵ , which does not depend on the network’s parameters — so gradients can flow through $\mu_z(x)$ and $\sigma_z(x)$ as usual.



VARIATIONAL AUTOENCODERS AS A GENERATIVE MODEL

- New data can be generated by sampling from distributions in the latent space, then reconstructing via the decoder.
- A diagonal prior enforces independent latent variables $\rightarrow$ can encode different factors of variation.

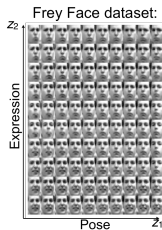


Figure: Images generated by a VAE, superimposed on the latent vectors that generated them: dimension 1 encodes face position, dimension 2 encodes expression (Goodfellow et al., 2016).

SAMPLES FROM A VANILLA VAE



Figure: Samples generated by a VAE trained on images of faces.

VARIATIONAL AUTOENCODERS:

SUMMARY

- Probabilistic model → allows generating data.
- Intractable density → optimizes the variational lower bound (ELBO) instead.
- Trained by backpropagation, using the reparameterization trick.
- Pros:
 - principled approach to generative models,
 - the latent-space reparameterization is useful for other tasks too.
- Cons:
 - only maximizes a *lower bound* of the likelihood,
 - samples in standard VAEs are often blurrier than samples from other modern generative approaches.
- Active area of research!

REFERENCES



Jordan, J. (2018). *Variational Autoencoders*.

<https://www.jeremyjordan.me/variational-autoencoders/>



Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Transformer Outlook

WHERE THIS IS GOING: ATTENTION AND TRANSFORMERS

- The encoder-decoder RNN squeezes an entire input sequence into a **single** fixed-length context vector C — Bahdanau et al. (2015) identify this as a bottleneck: the network must compress all information about a source sentence, regardless of its length, into one vector.
- Empirically, this bottleneck bites hard on long sentences: on the WMT'14 English-to-French task, a plain encoder-decoder's BLEU score collapses as sentence length grows past ~ 20 words, while an attention-based model's stays essentially flat even past 50 words (Bahdanau et al., 2015).

WHERE THIS IS GOING: ATTENTION AND TRANSFORMERS / 2

- **Attention** fixes this by letting the decoder look back at *all* encoder hidden states at every decoding step, learning which parts of the input matter most for the current output — rather than compressing everything into one vector up front.
- This turns out to be a more fundamental idea than just an RNN add-on: Vaswani et al. (2017) show an architecture “based solely on attention mechanisms, dispensing with recurrence and convolutions entirely” — and it set a new state of the art (28.4 BLEU on WMT’14 English-German, +2 BLEU over the best prior ensemble; 41.8 BLEU on English-French) while training in a fraction of the time of prior models.

WHERE THIS IS GOING: ATTENTION AND TRANSFORMERS / 3

- That architecture is the **Transformer** — the subject of Session 7, in depth.

WHY THIS MATTERS FOR GENERATIVE MODELS TOO

- Every generative model family in this session — autoencoders, VAEs — is defined by *what* is being learned (a compressed code, a latent distribution), independent of *which* network computes it.
- Any of them can use an RNN, a CNN, or a Transformer as their encoder/decoder — the architectures in this session and CNNs (Session 5) are building blocks, not fixed pairings.
- In practice, Transformer-based encoders and decoders have now displaced RNNs in most large-scale sequence and generative models.

REFERENCES



Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. Published as a conference paper at ICLR 2015. <https://arxiv.org/abs/1409.0473>



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. <https://arxiv.org/abs/1706.03762>

License & Attribution

LICENSE & ATTRIBUTION

- RNN sections (Motivation and Mechanics; Worked Example and Use Cases; Computational Graph and Bidirectional RNNs; Backpropagation Through Time; LSTM and GRU; RNN Applications), Unsupervised Learning, Autoencoders (Basic Principle; Regularized Autoencoders), and Variational Autoencoders adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, SLDS, LMU Munich.
https://github.com/slds-lmu/lecture_i2dl — licensed CC BY 4.0:
<https://creativecommons.org/licenses/by/4.0/>

LICENSE & ATTRIBUTION / 2

- The “Multi-Layer (Stacked) RNNs” frame is adapted from **Deep Learning for NLP (DL4NLP)**, SLDS, LMU Munich — also licensed **CC BY 4.0**. This topic is not covered in I2DL’s own RNN material; its diagram was redrawn independently rather than reusing the original’s custom TikZ macros.
- The “Transformer Outlook” bridge is original text written for this course, not adapted from I2DL, DL4NLP, or any other source.
- Content in this deck has been merged, reordered, and (for VAE derivations) condensed from the originals; not used verbatim.